1. Sebuah komputer memiliki 8 core CPU. Sebuah program membuat 16 thread untuk memproses data secara paralel.

a. Apakah 16 thread dapat berjalan secara bersamaan? Jelaskan dengan mempertimbangkan jumlah core dan mekanisme penjadwalan OS.

Tidak sepenuhnya, dengan 8 core, hanya 8 thread yang benar-benar berjalan secara paralel pada satu waktu (asumsi tanpa hyper-threading). Sisa 8 thread lainnya menunggu di ready queue.

Scheduler OS menggunakan time slicing (preemptive multitasking), dimana setiap thread diberi jatah waktu (quantum) di core, lalu di-preempt dan digantikan thread lain lewat context switch. Sehingga 16 thread terlihat berjalan bersamaan (konkuren) tetapi secara fisik hanya 8 yang paralel

Jika CPU mendukung hyperthreading/SMT, maka 8 core fisik bisa menyediakan 16 hardware thread. Namun, dua thread dalam 1 core berbagi unit eksekusi, sehingga performanya tidak setara 16 core penuh.

b. Apa yang mungkin terjadi jika jumlah thread jauh lebih banyak daripada jumlah core?

1. Overhead context switch meningkat (menyimpan/memuat register, program counter, dll.)
2. Cache thrashing: working set tiap thread saling menggusur isi cache, sehingga cache miss naik
3. Konsumsi memori naik karena tiap thread punya stack sendiri
4. Kontensi pada lock atau sumber daya bersama meningkatn
5. Scheduler bekerja dengan lebih berat dan waktu tunggu setiap thread bertambah

c. Apakah peningkatan jumlah thread selalu meningkatkan performa? Berikan contoh kondisi ketika performa justru menurun.

Tidak, performa bisa turun ketika:

1. **Thread melebihi core pada tugas CPU-bound** (tugas yang waktunya habis untuk menghitung, bukan menunggu I/O) → jumlah core adalah batas kapasitas komputasi. Menambah thread di atas jumlah core tidak menambah kapasitas melainkan menambah biaya.
2. **Kontensi lock tinggi**: thread lebih banyak menghabiskan waktu menunggu mutex daripada bekerja
3. **Bottleneck memori I/O**: thread tambahan hanya memperebutkan sumber daya yang sama
4. **False sharing**: thread menulis variabel berbeda yang berada di cache line yang sama
5. **Hukum amdahl**: bagian serial program membatasi speedup maksimum

2. Perhatikan operasi berikut:

```
int counter = 0;

// Dilakukan oleh beberapa thread

counter = counter + 1;
```

a. Jelaskan mengapa kode tersebut dapat menimbulkan race condition.

Operasi tidak atomik. Di level mesin, ia terdiri dari tiga langkah:

1. LOAD counter -> register

2. ADD register, 1
3. STORE register -> counter

Jika dua thread menyisip di antara langkah-langkah ini, akan terjadi lost update.

Waktu	Thread A	Thread B	counter
1	load (0)		0
2		load (0)	0
3	add -> 1		0
4		add -> 1	0
5	store 1		1
6		store 1	**1**

Dua increment dilakukan, tetapi hasilnya hanya bertambah 1. Hasil akhir bergantung pada urutan eksekusi yang tidak deterministik sehingga terjadi race condition.

b. Jika empat thread masing-masing melakukan operasi increment sebanyak 1.000 kali, berapa nilai akhir yang diharapkan secara logis? Mengapa hasil aktual bisa berbeda?

Secara logis, diharapkan nilai akhir 4000. Namun, nilai aktual bisa kurang dari 4000 dan berubah-ubah setiap eksekusi) akibat masalah lost update yang telah dijelaskan di atas ⟶ sebagian increment tertimpa oleh thread lain yang membaca nilai lama

c. Jelaskan dua cara untuk memperbaiki masalah tersebut menggunakan mekanisme sinkronisasi.

1. **Mutex (mutual exclusion)**

```
pthread_mutex_lock(&lock);
counter = counter + 1;
pthread_mutex_unlock(&lock);
```

Hanya satu thread yang boleh berada di critical section pada satu waktu, sehingga urutan load-add-store tidak terpotong

2. **Operasi atomik (semaphore)**

```
atomic_fetch_add(&counter, 1);
```

Instruksi hardware menjamin read-modify-write berlangsung tak terpisahkan, tanpa lock dan dengan overhead lebih kecil.

Alt: #pragma omp critical/atomic, semaphore biner

3. Analisis Kode OpenMP

Perhatikan kode berikut:

```
#include <stdio.h>
```

```
#include <omp.h>
int main() {
  int sum = 0;
  #pragma omp parallel for
  for (int i = 1; i <= 1000; i++) {
    sum += i;
  }
  printf("Sum = %d\n", sum);
  return 0;
}
```

Jawablah pertanyaan berikut:

a. Apa tujuan penggunaan parallel for pada kode tersebut?

Tujuan #pragma omp parallel for adalah untuk membagi iterasi loop (1 .. 1000) ke beberapa thread agar dikerjakan secara **paralel**. Tiap thread mendapat sebagian rentang iterasi, sehingga waktu eksekusi total diharapkan berkurang.

b. Identifikasi kesalahan atau potensi masalah pada implementasi di atas.

Variabel sum bersifat **shared**. Semua thread melakukan sum += i pada variabel yang sama tanpa sinkronisasi, sehingga terjadi race condition (lost update). Hasilnya hampir pasti tidak sama dengan nilai yang sesungguhnya dan berbeda-beda pada setiap run.

c. Perbaiki kode agar menghasilkan nilai yang benar tanpa menghilangkan paralelisme.

Menggunakan reduction:

```
#include <stdio.h>
#include <omp.h>

int main() {
  int sum = 0;
  #pragma omp parallel for reduction(+:sum)
  for (int i = 1; i <= 1000; i++) {
    sum += i;
  }
  printf(“Sum = %d\n”, sum);
  return 0;
}
```

reduction(+:sum) membuat setiap thread memiliki salinan private sum (diinisialisasi 0). Tiap thread menjumlahkan bagiannya sendiri tanpa kontensi, lalu di akhir semua salinan digabung dengan operator + ke

variabel sum asli. Hal ini lebih baik daripada critical/atomic yang menserialisasi setiap penambahan.

Aspek	`reduction`	`atomic`	`critical`
Mekanisme	Salinan privat, digabung di akhir	Instruksi atomik per update	Lock untuk satu blok
Cakupan	Satu variabel akumulasi	Satu pernyataan sederhana	Blok kode bebas
Thread saling menunggu saat loop	Tidak	Ya (ringan)	Ya (berat)
Overhead	Terendah	Sedang	Tertinggi
Skalabilitas	Sangat baik	Terbatas	Buruk

4. Penjadwalan Iterasi OpenMP

Sebuah program menggunakan OpenMP untuk memproses 10.000 elemen array. Waktu pemrosesan setiap elemen berbeda-beda karena kompleksitas datanya tidak seragam.

Bandingkan strategi static, dynamic, dan guided pada klausa schedule.

a. Bagaimana masing-masing strategi membagikan iterasi kepada thread?

Strategi	**Cara kerja**
static	Iterasi dibagi di awal (saat kompilasi/mulai loop) menjadi chunk berukuran tetap dan ditetapkan ke thread secara round-robin. Tanpa chunk, iterasi dibagi rata menjadi blok kontigu. Penetapan tidak berubah saat runtime.
dynamic	Iterasi dibagi menjadi chunk berukuran sama dan kecil (default: 1 iterasi per chunk). Thread yang selesai meminta chunk berikutnya dari antrean saat runtime.
guided	Mirip dynamic, tetapi ukuran chunk dimulai besar lalu mengecil secara bertahap (sebanding dengan sisa iterasi/jumlah thread) hingga minimum yang ditentukan.

b. Apa trade-off antara overhead penjadwalan dan load balancing?

- **static**:
  - **overhead penjadwalan terendah**, karena tidak ada koordinasi saat runtime dan lokalitas cache baik
  - **load balancing buruk** jika beban per iterasi tidak seragam. Thread yang mendapat elemen berat akan menjadi bottleneck , sementara thread lainnya idle/nganggur
- **dynamic**:
  - **overhead penjadwalan tertinggi**, karena setiap pengambilan chunk mengharuskan

sinkronisasi pada antrean. Chunk yang terlalu kecil membuat overhead mendominasi, sedangkan chunk yang terlalu besar mengurangi kemampuan balancing.
    - **load balancing terbaik** karena thread yang cepat mengambil kerja lebih banyak (less idle time)
- **guided**:
    - chunk besar di awal menekan overhead, chunk kecil di akhir memperbaiki load balancing.
    - Overhead lebih rendah dari dynamic namun kurang optimal bila iterasi berat terkonsentrasi di awal loop, karena chunk awal yang besar bisa membuat satu thread kelebihan beban.

5. Perhatikan kode berikut:

```
#include <stdio.h>

#include <pthread.h>

int counter = 0;

void* increment(void* arg) {

  for (int i = 0; i < 100000; i++) {

    counter++;

  }

  return NULL;

}

int main() {

  pthread_t t1, t2;

  pthread_create(&t1, NULL, increment, NULL);

  pthread_create(&t2, NULL, increment, NULL);

  pthread_join(t1, NULL);

  pthread_join(t2, NULL);

  printf("Counter = %d\n", counter);

  return 0;

}
```

Jawablah pertanyaan berikut:

a. Berapa nilai akhir counter yang diharapkan jika seluruh operasi increment berhasil dilakukan?

2 thread x 100000 = 200000

b. Mengapa nilai aktual belum tentu sama dengan nilai yang diharapkan?

counter++ bukan operasi atomik (load, increment, store). Saat t1 dan t2 berjalan bersamaan pada variabel global counter, kedua thread dapat membaca nilai yang sama, lalu menimpa hasil satu sama lain (lost update). Hasil akhir biasanya di bawah 200000 dan berbeda setiap eksekusi.

c. Apa dampak penggunaan mutex terhadap performa jika setiap thread melakukan jutaan operasi increment?

- Menjamin kebenaran tetapi menurunkan performa karena critical section menserialisasi eksekusi → thread saling menunggu, sehingga paralelisme hilang
- operasi lock/unlock dipanggil jutaan kali. → Overhead terakumulasi besar apalagi jika terjadi kontensi